

1 INTRODUÇÃO

O desenvolvimento de sistemas computacionais envolve diferentes tipos de requisitos que precisam ser considerados ao longo do processo de construção do software. Alguns desses requisitos estão diretamente relacionados às funcionalidades que o sistema deve oferecer, enquanto outros estão relacionados a características que atravessam diferentes partes do sistema, como comportamento temporal, controle de acesso a recursos compartilhados, consumo de energia e outras propriedades associadas ao funcionamento do sistema. Em sistemas embarcados de tempo real, essa distinção torna-se particularmente relevante, uma vez que tais sistemas precisam atender simultaneamente às funcionalidades previstas e a restrições relacionadas ao seu comportamento e à plataforma na qual serão executados.

Uma das dificuldades associadas a esse tipo de desenvolvimento está em representar essas diferentes características de maneira organizada e suficientemente precisa desde as etapas iniciais do projeto. Quando determinados requisitos estão relacionados a diversos elementos do sistema, sua implementação pode envolver diferentes partes do software. Como consequência, uma mesma característica pode aparecer distribuída em vários elementos, dificultando sua identificação, manutenção e reutilização. Esse problema está relacionado ao tratamento das chamadas características transversais, isto é, características que não estão necessariamente restritas a um único elemento do sistema.

Uma alternativa para lidar com essa complexidade consiste em trabalhar em níveis mais elevados de abstração, utilizando modelos como elementos centrais do processo de desenvolvimento. Nesse contexto, a Engenharia Orientada a Modelos, ou Model-Driven Engineering (MDE), utiliza modelos para representar o sistema em diferentes níveis de abstração e permite que esses modelos participem das etapas de desenvolvimento até a geração da implementação. A utilização de modelos busca, entre outros aspectos, reduzir a dependência de detalhes de implementação nas fases iniciais do projeto e possibilitar transformações sistemáticas entre diferentes representações do sistema. No contexto de sistemas embarcados de tempo real, essa estratégia é particularmente relevante porque permite que características do domínio sejam representadas antes que o projetista precise lidar diretamente com detalhes da plataforma de execução.

Entretanto, somente a utilização de modelos não resolve, por si só, o problema da representação de características que atravessam diferentes partes do sistema. Por essa razão, conceitos do Desenvolvimento Orientado a Aspectos, ou Aspect-Oriented (AO), podem ser associados à Engenharia Orientada a Modelos. A orientação a aspectos procura possibilitar a modularização de determinadas características que, de outra forma, poderiam ficar distribuídas por diferentes partes do sistema. A combinação desses conceitos permite trabalhar com os requisitos funcionais e não funcionais em níveis mais altos de abstração e, ao mesmo tempo, representar características transversais de maneira modular.

É nesse contexto que Wehrmeister propõe a abordagem Aspect-oriented Model-Driven Engineering for Real-Time systems (AMoDE-RT), voltada ao desenvolvimento de sistemas embarcados de tempo real. A AMoDE-RT combina conceitos de Engenharia Orientada a Modelos e de orientação a aspectos com o objetivo de aumentar o nível de abstração durante o projeto e permitir um tratamento modular dos requisitos funcionais e não funcionais. Na abordagem, modelos UML são utilizados para representar os elementos do sistema, enquanto conceitos orientados a aspectos são empregados para representar determinadas características que atravessam esses elementos.

Para compreender o funcionamento dessa abordagem, é necessário primeiro compreender o papel do modelo UML. De maneira simplificada, o modelo UML pode ser entendido como uma representação estruturada do sistema que está sendo desenvolvido. Em vez de iniciar diretamente pelo código-fonte, o projetista descreve aspectos da estrutura e do comportamento do sistema por

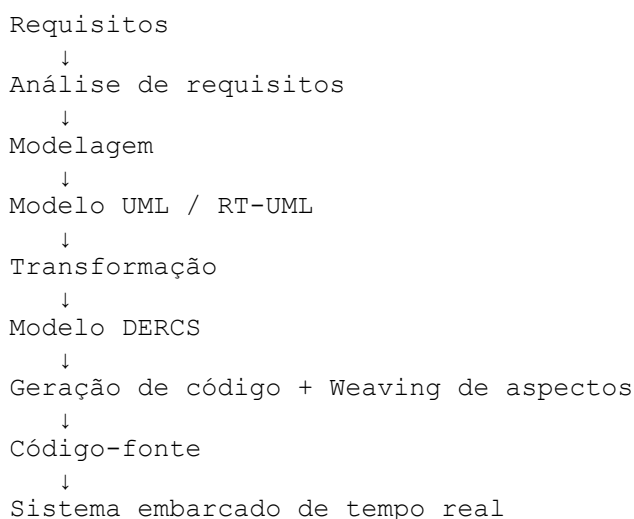
meio de elementos de modelagem. No contexto da AMoDE-RT, esses modelos são sucessivamente refinados até atingir um nível de detalhe considerado suficiente para a realização do sistema. Os elementos inicialmente representados podem estar próximos dos conceitos do domínio da aplicação, como sensores, dispositivos de controle, braços robóticos ou outras entidades relacionadas ao sistema que está sendo desenvolvido.

Para tratar os requisitos não funcionais relacionados aos sistemas embarcados de tempo real, a AMoDE-RT utiliza o Distributed Embedded Real-time Aspects Framework (DERAF). O DERAf fornece aspectos de alto nível destinados ao tratamento de um conjunto de requisitos não funcionais encontrados nesse domínio. Esses aspectos podem acrescentar comportamento e estrutura ao modelo para representar como determinados requisitos não funcionais devem ser tratados. Entre os exemplos apresentados por Wehrmeister estão aspectos relacionados a ativação periódica de tarefas, controle de acesso a recursos compartilhados e monitoramento de consumo de energia.

Nesse ponto surge um conceito fundamental para esta dissertação: o conceito de *Join Point*. Quando um aspecto precisa afetar determinados elementos de um modelo, é necessário estabelecer quais elementos serão atingidos por suas adaptações. Os *Join Points* estão relacionados justamente à seleção dos elementos que serão afetados pelas adaptações dos aspectos. Portanto, um *Join Point* pode ser compreendido, neste contexto, como uma indicação dos elementos do modelo sobre os quais determinada característica transversal deverá atuar.

Essa seleção é importante porque o aspecto, por si só, não determina necessariamente todos os elementos sobre os quais deverá atuar. É necessário estabelecer um critério que permita identificar esses elementos. Na abordagem de Wehrmeister, os *Join Points* são especificados utilizando *Join Point Designation Diagrams* (JPDD). O JPDD é utilizado para representar os critérios de seleção dos elementos que serão afetados pelas adaptações dos aspectos.

A partir dessa especificação, a AMoDE-RT possui uma sequência de transformações que conduz o modelo de mais alto nível até a geração do código. Uma representação simplificada desse processo pode ser apresentada da seguinte maneira:



Essa sequência é relevante para o presente trabalho porque mostra que a seleção dos elementos afetados por características transversais não é uma atividade isolada. Ela faz parte de um processo maior de transformação do modelo e geração da implementação. Na AMoDE-RT, o modelo UML é transformado em uma representação intermediária denominada Distributed Embedded Real-Time Compact Specification (DERCS). O DERCS foi proposto como uma representação mais concisa do modelo, adequada às etapas posteriores de geração de código e execução do modelo, além de possuir elementos próprios para representar conceitos orientados a aspectos, como aspectos, *pointcuts* e *join points*.

A transformação entre UML e DERCS é realizada automaticamente pela ferramenta Generation of

Embedded Real-Time Code based on Aspects (GenERTiCA). Essa ferramenta foi desenvolvida para apoiar a AMoDE-RT e realiza tanto a geração de código quanto o *weaving* de aspectos. De forma simplificada, o *weaving* pode ser entendido como o processo pelo qual as adaptações associadas aos aspectos são incorporadas aos fragmentos de código correspondentes aos elementos que foram identificados como afetados pelo aspecto. Assim, a ferramenta percorre os elementos do modelo DERCS, identifica as regras de geração correspondentes e, quando um elemento é afetado por um aspecto, incorpora ao código gerado as respectivas adaptações.

O DERCS desempenha, portanto, um papel intermediário nesse processo. Ele recebe as informações provenientes do modelo UML e as organiza em uma representação que pode ser utilizada para a geração do código. No processo descrito por Wehrmeister, os elementos relacionados aos critérios de seleção também participam dessa transformação. Modelos que contêm especificações de JPDD podem ser transformados em elementos *JoinPoint* do DERCS, sendo selecionados os elementos que correspondem aos critérios de busca definidos na especificação.

Embora esse mecanismo permita estabelecer a relação entre a especificação e os elementos selecionados, a forma utilizada para representar os critérios de seleção também constitui uma questão importante. Posteriormente, Oliveira investigou a especificação das ocorrências de características transversais no contexto da AMoDE-RT. Seu trabalho parte da utilização dos JPDDs e propõe uma nova técnica denominada *Join Point Specification Diagram* (JSD). A JSD foi proposta para ser utilizada no processo de modelagem da AMoDE-RT, substituindo o JPDD, a partir de uma preocupação com a compreensão da especificação dos pontos de ocorrência de características transversais.

A proposta da JSD introduz uma característica especialmente relevante para o presente trabalho: a utilização da linguagem Object Constraint Language (OCL) para expressar os critérios de seleção. Na JSD, elementos gráficos são utilizados em conjunto com elementos textuais para tornar explícitos os critérios empregados na busca dos pontos de ocorrência de características transversais. A OCL é utilizada justamente para descrever esses critérios de seleção sobre os elementos do modelo UML.

Para compreender a importância dessa utilização da OCL, é necessário distinguir duas ideias: especificar uma seleção e efetivamente realizar essa seleção.

Uma especificação de seleção descreve uma regra que informa quais características um elemento deve possuir para ser considerado correspondente a determinado critério. Em termos simplificados, pode-se imaginar uma regra como: “selecione os elementos que possuem determinada característica”. A expressão OCL permite representar esse tipo de critério de maneira formal e explícita. No contexto da JSD, portanto, a OCL participa da descrição daquilo que deve ser procurado no modelo.

Entretanto, existe uma questão adicional quando se pretende utilizar essa especificação de maneira computacional: não basta possuir uma expressão que descreva o critério. É necessário que exista um mecanismo capaz de receber essa expressão, interpretá-la em relação ao modelo UML e produzir como resultado os elementos que satisfazem o critério.

Essa distinção constitui o ponto de partida desta dissertação.

Enquanto a especificação define **o que deve ser selecionado**, a execução precisa determinar **quais elementos concretos do modelo correspondem a essa especificação**. Em outras palavras, existe uma diferença entre escrever uma expressão OCL que representa um critério de seleção e executar essa expressão sobre um modelo UML para obter os elementos selecionados.

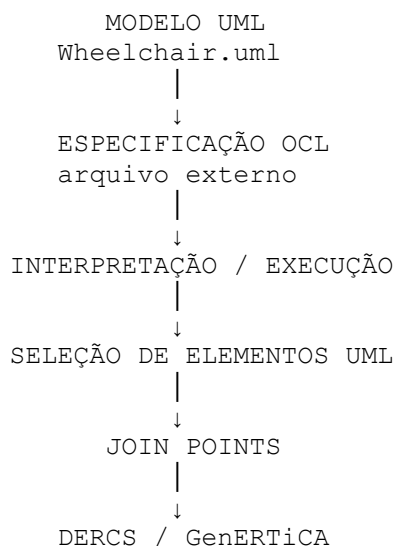
É nessa etapa que se concentra o presente trabalho.

Nesta dissertação, investiga-se a utilização de uma expressão OCL externa para realizar a seleção de elementos de um modelo UML, tendo como estudo de caso o modelo *Wheelchair.uml*. O objetivo é estabelecer um mecanismo capaz de carregar o modelo UML, interpretar uma especificação OCL e

identificar os elementos do modelo que satisfazem os critérios definidos pela expressão.

O uso de uma expressão OCL externa significa, nesse contexto, separar a especificação da consulta do programa responsável por executá-la. Em vez de manter toda a expressão diretamente no código Java, a expressão é armazenada em um arquivo próprio e posteriormente carregada pelo protótipo. Essa separação permite investigar de maneira mais direta o processo de leitura, interpretação e execução de uma especificação OCL sobre um modelo UML.

O fluxo investigado neste trabalho pode ser representado conceitualmente da seguinte maneira:



Nesse fluxo, é importante observar que as últimas etapas representam uma possibilidade de utilização posterior dos elementos selecionados. O foco efetivamente investigado nesta dissertação está na etapa que parte do modelo UML e da expressão OCL e chega à identificação dos elementos correspondentes. A integração completa desses elementos com DERCS e GenERTiCA não é assumida como uma etapa já implementada, mas como uma possível continuidade do processo.

Para investigar essa questão, foi desenvolvido um protótipo utilizando Java, Eclipse, EMF/UML2 e Eclipse OCL. O EMF/UML2 é utilizado para trabalhar programaticamente com o modelo UML, permitindo carregar o arquivo *Wheelchair.uml* e acessar seus elementos. O ambiente Eclipse OCL é utilizado para fornecer os mecanismos relacionados à linguagem OCL, incluindo a criação de consultas e a tentativa de avaliação das expressões sobre o modelo carregado.

O protótipo foi estruturado de forma a separar as diferentes responsabilidades envolvidas no processo. O carregamento do modelo é realizado de maneira independente da leitura do arquivo contendo a expressão OCL. Posteriormente, a expressão é analisada e encaminhada para o ambiente OCL, que deve ser capaz de interpretá-la considerando os tipos e elementos existentes no modelo UML.

Até o estágio atual da investigação, foi possível carregar o modelo *Wheelchair.uml*, acessar seus elementos, inicializar o ambiente OCL, criar o `OCLHelper`, realizar testes com `allInstances()` e processar expressões OCL armazenadas em arquivo externo.

Entretanto, os experimentos também evidenciaram uma dificuldade importante. Durante a tentativa de executar as expressões sobre o modelo, foi identificado um problema relacionado à resolução de operações de coleção pelo ambiente Eclipse OCL utilizado. Uma expressão mínima como:

```
self.allOwnedElements()
```

já provoca uma `ParserException` durante a criação da consulta, mesmo sem a utilização de operadores mais complexos, como `select`.

Esse comportamento é relevante para a investigação porque a expressão utilizada constitui justamente uma operação básica para percorrer os elementos pertencentes ao modelo e, a partir deles, realizar posteriormente operações de seleção. Se a resolução dessa operação não ocorre corretamente, a execução de expressões mais completas de seleção também fica comprometida.

Dessa forma, o problema investigado não se limita à construção de uma expressão OCL. É necessário compreender também como o ambiente de execução interpreta essa expressão, como identifica os tipos envolvidos, como resolve as operações disponíveis para esses tipos e em que ponto ocorre a falha. A análise do erro indica a participação de componentes responsáveis pela criação e resolução de tipos e operações de coleção da OCL. Assim, a investigação passa a envolver não apenas a expressão em si, mas também as condições necessárias para que ela seja corretamente interpretada pelo ambiente utilizado.

Esse aspecto é particularmente importante porque a expressão OCL pretendida para a seleção dos elementos possui uma estrutura mais elaborada. A ideia é partir dos elementos do modelo e aplicar critérios sucessivos para restringir a seleção até chegar aos elementos que correspondem ao *Join Point* especificado. Conceitualmente, o processo pode ser representado como:

```
Elementos do modelo
    ↓
seleção dos elementos de interesse
    ↓
aplicação dos critérios OCL
    ↓
refinamento da seleção
    ↓
elementos correspondentes ao Join Point
```

Portanto, a execução de uma expressão como `self.allOwnedElements()` representa uma etapa preliminar desse processo. Antes que seja possível investigar completamente expressões que utilizem operações como `select`, é necessário compreender por que uma operação básica de acesso aos elementos do modelo já apresenta uma falha no ambiente utilizado.

A partir desse contexto, esta dissertação busca contribuir para a compreensão e implementação do processo de execução de especificações OCL utilizadas na seleção de elementos de modelos UML. O trabalho procura estabelecer uma ligação entre a especificação de *Join Points* no contexto da JSD, na qual a OCL é utilizada para expressar os critérios de seleção, e sua efetiva interpretação computacional sobre um modelo UML.

Essa ligação pode ser compreendida a partir de três níveis. No primeiro nível está a **especificação**, na qual se define por meio da OCL quais características devem ser consideradas na seleção. No segundo está a **execução**, responsável por interpretar essa especificação em relação a um modelo concreto. No terceiro está o **resultado da seleção**, constituído pelos elementos do modelo que satisfazem os critérios definidos. Esse resultado pode, posteriormente, ser relacionado ao conceito de *Join Point* e às etapas seguintes da abordagem, envolvendo DERCS e GenERTiCA.

Assim, o objetivo desta investigação não é propor novamente uma técnica de especificação de *Join Points*, uma vez que a JSD já constitui uma proposta existente nesse contexto. Também não se pretende assumir como implementada a transformação completa para DERCS ou a geração e o *weaving* de código pelo GenERTiCA. O foco está em uma etapa anterior e necessária: compreender e implementar a execução da especificação OCL sobre o modelo UML, de modo que os elementos correspondentes aos critérios possam ser efetivamente identificados.

O estudo de caso baseado no modelo *Wheelchair.uml* permite investigar essa questão em um modelo concreto, possibilitando acompanhar o processo desde o carregamento do modelo até a tentativa de execução da expressão OCL. A partir dos experimentos realizados, são analisados tanto os mecanismos necessários para realizar a seleção quanto as limitações encontradas no ambiente Eclipse OCL utilizado.

Desse modo, a investigação estabelece uma continuidade entre os trabalhos anteriormente desenvolvidos no contexto da AMoDE-RT e a necessidade de execução computacional dos critérios de seleção. A trajetória pode ser resumida da seguinte forma: a AMoDE-RT estabelece uma abordagem baseada em modelos e aspectos; o DERAf fornece mecanismos para tratar requisitos não funcionais; os *Join Points* identificam os elementos afetados pelas adaptações; os JPDDs foram utilizados para especificar esses pontos; a JSD foi posteriormente proposta como uma nova forma de especificação, utilizando OCL para tornar explícitos os critérios de seleção; e o presente trabalho concentra-se na investigação de como esses critérios expressos em OCL podem ser efetivamente interpretados e executados sobre um modelo UML.

Nesse sentido, o trabalho procura avançar de uma especificação que **descreve quais elementos devem ser selecionados** para um mecanismo que **identifica efetivamente esses elementos no modelo**. Os resultados obtidos nessa etapa podem constituir uma base para investigações posteriores relacionadas à utilização dos elementos selecionados no processo de transformação para DERCS e nas etapas de geração e adaptação de código associadas ao GenERTiCA, sem considerar essas etapas como concluídas no escopo desta investigação.